



LINGUAGEM DE PROGRAMAÇÃO

AULA 6



Prof. Wellington Rodrigo Monteiro



CONVERSA INICIAL

Olá! Nesta aula, temos como objetivo mostrar a você alguns tópicos adicionais sobre o desenvolvimento de bons algoritmos. É comum acreditarmos que o processo do desenvolvimento de software se resume tão somente a **resolver problemas**. Até chegamos a destacar esse ponto anteriormente: criamos **soluções** com o uso de algoritmos, e não algoritmos sem um porquê de eles existirem em primeiro lugar. Contudo, acabamos ignorando um segundo aspecto tão importante quanto a resolução de problemas: a **sustentabilidade**.

Do ponto de vista de Tecnologia da Informação, a sustentabilidade não somente cobre o desenvolvimento de softwares visando à redução do consumo de energia e/ou a solução de problemas que afetam o meio ambiente, mas também a **sustentação** da solução em si: em bom português, a garantia de que aquela solução irá “parar em pé” no futuro. Este é um bom desafio já que, dependendo do cenário, acabamos testando os nossos algoritmos contra pequenos casos de teste visando tão somente à validação da lógica, mas ignorando a performance quando a base a ser processada é suficientemente grande.

Nesse contexto, a aula atual buscará explicar para você conteúdos como otimização, concorrência, escalabilidade e interfaces (também conhecidos como *protocolos* dependendo da linguagem de programação). Vamos lá?

TEMA 1 – OTIMIZAÇÃO

Se você estiver lendo este texto a partir de qualquer computador ou celular, perceberá que existem várias outras atividades sendo executadas em paralelo: pode ser que o seu computador esteja também tocando alguma música; rodando algum processo para atualizar o relógio e a previsão do tempo; outro para rodar um firewall; outro para rodar um antivírus; outro para gerenciar chats, e assim por diante. Nos smartphones, é praticamente a mesma coisa: existem inúmeros apps em execução, e um não pode esperar vários minutos para que o outro termine o seu trabalho: todos acabam rodando praticamente ao mesmo tempo. Pode ser que você esteja assistindo a um vídeo e, enquanto isso, outro app de troca de mensagens obterá as últimas atualizações e um app de delivery lhe avisará sobre as últimas ofertas.



Logo, observe aqui que sempre existem várias tarefas sendo executadas **em paralelo** em um computador ou celular atuais. É esse paralelismo que permite que tenhamos múltiplas janelas (e/ou abas) abertas ao mesmo tempo ou, ainda, muitos aplicativos abertos ao mesmo tempo. Por outro lado, estamos pensando no lado do computador ou do celular: eles **possuem suporte** para isso e os seus sistemas (seja Android, Mac OS, Windows, Linux ou qualquer outro sistema atual) foram arquitetados para fazer isso da melhor forma possível.

Dito isso, pensemos: será que o mesmo realmente se aplica para os softwares que são desenvolvidos para esses sistemas também? Será que todos os softwares desenvolvidos são muito otimizados e funcionam sem problemas independentemente do hardware ou do contexto em que estão sendo executados? Essas perguntas são retóricas, mas vamos entender isso melhor na prática: você nunca se deparou com algum aplicativo que consome muita memória sem nenhuma justificativa aparente? Ou, ainda, um jogo que é muito lento apesar de você ter consciência que o seu computador dá conta? Ou um navegador que consome memória e processador em excesso? Ou um aplicativo no seu celular que é pesado demais e consome muita bateria sem aparentemente precisar disso?

Esses casos poderiam ser descritos como sendo o de softwares que poderiam ser melhor **otimizados**. O termo *otimização* é amplo e pode significar várias coisas: na área da Engenharia de Produção, poderia ser maximizar ou minimizar algo (como gerar uma rota para que um caminhão faça o máximo de entregas no menor tempo possível). No ponto de vista de software, isso pode indicar outra coisa: pode ser tanto a melhoria da **eficiência** do algoritmo como dos processos.

1.1 Otimização de processos

Neste caso, a otimização aplica-se ao que uma organização está fazendo com a **gestão**: existem organizações que podem ter um software muito bom, mas praticamente sem controles de Governança de Tecnologia da Informação por trás, ou sem nenhuma segurança sobre o que está sendo desenvolvido. Imagine que nós queremos construir uma casa: sabemos que não é **obrigatório** um engenheiro civil e um arquiteto para construir essa casa. Na realidade, essas duas funções não construiriam com as suas mãos, mas planejariam as necessidades da casa para que ela fosse construída seguindo as normas e



demais requisitos para aquele determinado local. Em outras palavras, poderíamos afirmar que uma ou duas pessoas exercendo o trabalho de um construtor ou pedreiro já poderiam construir a casa. De fato, somente essas pessoas fariam todo o trabalho mais rápido e mais barato: mais rápido porque não teríamos que pedir a aprovação de ninguém para construir e também porque não precisaríamos criar primeiro um projeto da construção. Mais barato porque não teríamos que pagar uma pessoa ocupando a função de engenheiro, arquiteto ou, ainda, algumas taxas da prefeitura.

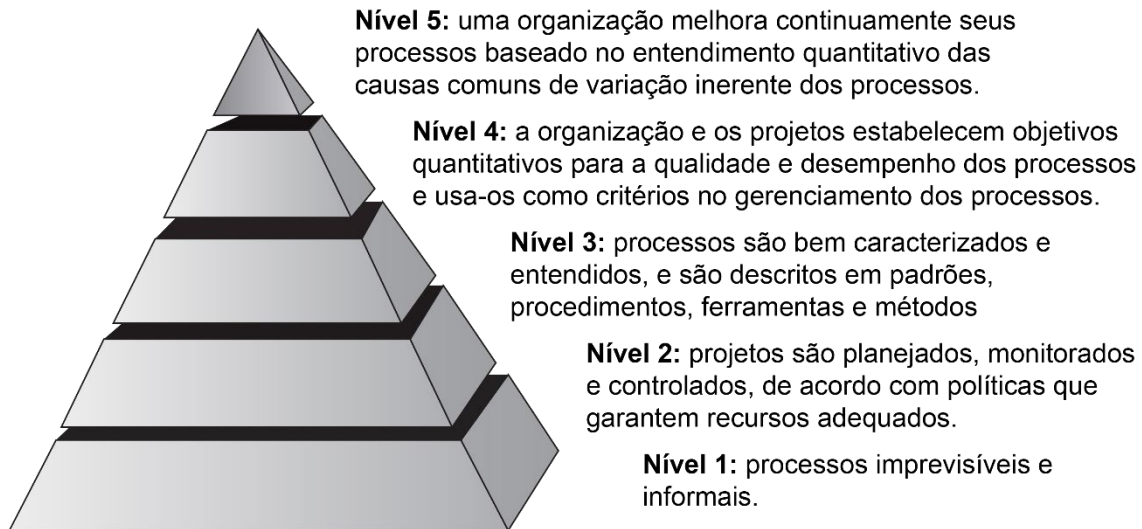
Por outro lado, isso não é sinônimo de **padrão** e nem de **boas práticas**: não estamos aqui afirmando que o construtor ou o pedreiro fariam um trabalho ruim por conta – longe disso! Contudo, não é a responsabilidade principal dessas funções o conhecimento das normas de trabalho e a legislação **atualizada** sobre a construção de casas. Também não é da sua responsabilidade a criação e a responsabilidade do projeto, a qual visa à segurança, à sustentabilidade e à qualidade da construção, entre outros aspectos. Logo, para que aquela casa seja de fato uma construção que siga todas as regras e boas práticas, precisamos necessariamente de uma pessoa que ocupe uma posição de engenheiro.

E por que estamos falando sobre isso? Ora, a mesma lógica se aplica ao software (e é por isso que o curso de Engenharia de Software existe em primeiro lugar): há uma diferença entre tão somente desenvolver um software de acordo com as próprias convicções e fazer aquilo que acha que está certo com o desenvolvimento de software seguindo normas técnicas, com um bom planejamento e seguindo boas práticas de desenvolvimento de código e a formalização dos processos de desenvolvimento de código.

Existem modelos que possuem como objetivo mensurar a maturidade de uma organização adquirida ao longo do tempo sob o ponto de vista de desenvolvimento de software. Desses, um dos mais conhecidos é o CMMI (*Capability Maturity Model Integration*), que é subdividido em três grupos: CMMI-DEV (desenvolvimento), CMMI-ACQ (aquisição) e CMMI-SVC (serviço). A pirâmide a seguir mostra os níveis de maturidade para o CMMI-DEV. O primeiro nível chama-se *Inicial*; o segundo chama-se *Gerenciado*; o terceiro chama-se *Definido*; o quarto chama-se *Quantitativamente Gerenciado* e o quinto e último chama-se *Em Otimização*. É este o termo que gostaríamos de reforçar aqui: esse nível trata da melhoria contínua dos processos de desenvolvimento de software com técnicas de controle estatístico. Aqui, há o gerenciamento das mudanças



de tecnologia e processos, bem como técnicas de prevenção de defeitos. Veja que estamos sempre discutindo aqui a melhoria dos **processos**, e não o código em si. Ou seja: não entramos no código-fonte, mas, sim, observamos **como** esse código-fonte é produzido. Logo, o foco é maior na gestão do que na execução.



1.2 Otimização de algoritmos

Por outro lado, também é possível pensar em otimização do ponto de vista dos algoritmos – isto é, na otimização do código-fonte. Existem algumas formas de mensurar isso, e a ideia é mostrar aqui alguns desses casos.

A primeira é ao analisar a eficiência do algoritmo: isto é, quantos cálculos um determinado algoritmo precisa fazer para finalizar uma tarefa. Esses cálculos, na prática, acabam traduzindo em uso do processador, energia elétrica gasta e, finalmente, no tempo investido. Existem algoritmos que rodam muito rápido no nosso computador para alguns testes, mas isso não significa necessariamente que será rápido com o dobro de dados sendo analisados (ou o triplo, ou cem vezes mais).

Uma das notações utilizadas para analisar esta eficiência é a Big O (ou notação assintótica, ou O Grande/Grande O). Ela mostra a performance do algoritmo conforme o tamanho dos dados sendo processados aumentam tendendo ao infinito.

Para ilustrar isso, pensemos em três funções escritas em Python, chamadas *operacao_constante*, *operacao_linear* e *operacao_quadratica*. Cada uma dessas funções faz uma operação diferente, mas todas elas recebem uma mesma lista:



- a função *operacao_constante* sempre calcula a metade do último número da lista *independentemente do tamanho da lista*;
- a função *operacao_linear* sempre calcula o dobro de cada valor da lista – ou seja, ele percorre *um valor de cada vez* da lista para fazer o cálculo; e
- a função *operacao_quadratica* multiplica todos os valores da lista contra todos os valores da mesma lista – ou seja, percorre cada valor da lista ao menos duas vezes para calcular o valor.

Todas as funções também retornam um *contador* para que saibamos quantos cálculos cada função fez. Assim, temos o seguinte:

```
# retorna a metade do último número
def operacao_constante(itens):
    contador = 1
    return contador, itens[-1]/2

# multiplica cada número da lista por dois
def operacao_linear(itens):
    contador = 0
    novos_itens = []
    for item in itens:
        novos_itens.append(item * 2)
        contador += 1

    return contador, novos_itens

# multiplica cada número da lista por todos os números da mesma lista
def operacao_quadratica(itens):
    contador = 0
    novos_itens = []
    for item in itens:
        for item_2 in itens:
            novos_itens.append(item * item_2)
            contador += 1

    return contador, novos_itens
```

Pensemos em um caso no qual temos uma lista de 10 números para vermos quantos cálculos cada função faz. Como não estamos interessados na lista resultante e só queremos o contador de cada função, utilizamos o underscore (“_”) para ignorar estas listas:



```
itens = list(range(10))

contador_constante, _ = operacao_constante(itens)
contador_linear, _ = operacao_linear(itens)
contador_quadratico, _ = operacao_quadratica(itens)

print(f'Contador constante: {contador_constante}')
print(f'Contador linear: {contador_linear}')
print(f'Contador quadrático: {contador_quadratico}')
```

```
Contador constante: 1
Contador linear: 10
Contador quadrático: 100
```

Veja que a operação constante fez somente 1 cálculo para essa lista de 10 elementos; a linear fez 10 cálculos e a quadrática fez 100 cálculos. E se mudássemos a lista para 25 elementos?

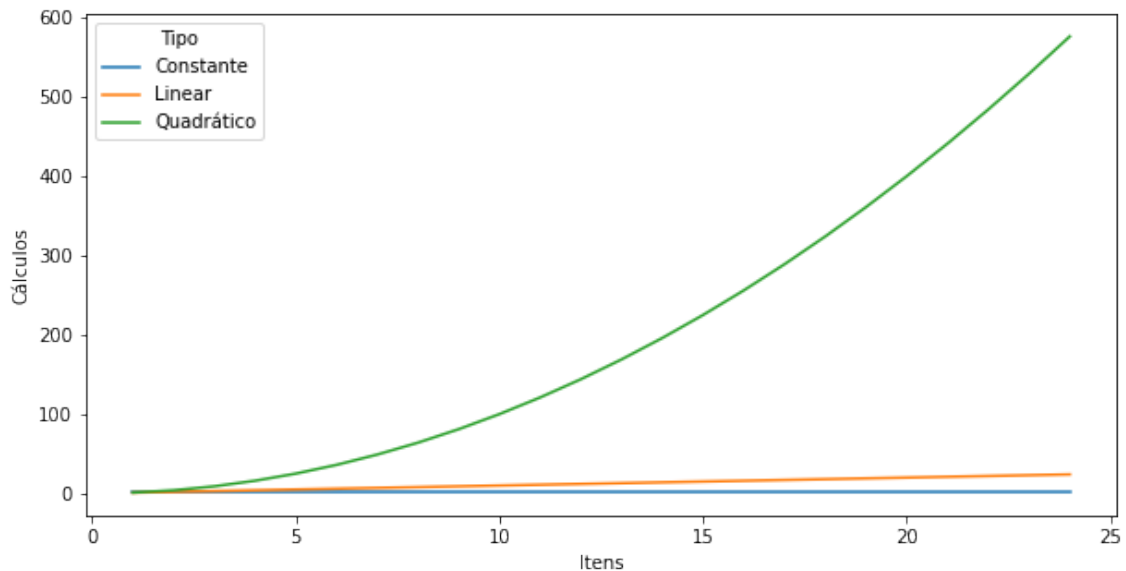
```
itens = list(range(25))

contador_constante, _ = operacao_constante(itens)
contador_linear, _ = operacao_linear(itens)
contador_quadratico, _ = operacao_quadratica(itens)

print(f'Contador constante: {contador_constante}')
print(f'Contador linear: {contador_linear}')
print(f'Contador quadrático: {contador_quadratico}')
```

```
Contador constante: 1
Contador linear: 25
Contador quadrático: 625
```

Veja que o resultado do contador linear permaneceu o mesmo. O linear aumentou e 10 para 25, mas o quadrático aumentou muito mais: de 100 para 625. Logo, se fôssemos aumentar de 25 para 100 itens, ou 1000 itens, é esperado que o contador linear aumente de forma linear e o quadrático aumente vertiginosamente. Vejamos como fica de forma visual.



Fonte: Monteiro, 2021.

A linha quadrática rapidamente cresce e destoa das demais. Já a linha da constante permanecerá com um único valor não importando a quantidade de itens sendo processados. Por outro lado, a linha linear cresce conforme a quantidade de itens aumenta – mas não em uma proporção tão grande quando em um algoritmo de complexidade quadrática. Quando estamos na Universidade, nos deparamos poucas vezes com problemas de otimização e complexidade algorítmica porque a quantidade de itens que manipulamos é baixa – por outro lado, imaginemos o seguinte: o eixo x do gráfico acima conta com 25 itens. Mas e se fossem 25 milhões?

O gráfico a seguir mostra outros tipos de complexidade: além da constante que já vimos anteriormente ($O(1)$, mostrada como “1” no gráfico a seguir), a linear ($O(n)$, mostrada como “n” no gráfico abaixo) e a quadrática ($O(n^2)$, mostrada como “n²” a seguir), temos também as complexidades fatorial ($O(n!)$), exponencial ($O(2^n)$), logarítmica linear ($O(n \log_2 n)$), raiz quadrada ($O(\sqrt{n})$) e logarítmica ($O(\log_2 n)$). Quanto mais próximo da constante, melhor; quanto mais próximo de uma complexidade fatorial, pior.

Algoritmos de força bruta (ou seja, que testam todas as combinações possíveis para achar a melhor solução) estão mais próximos de um problema de complexidade fatorial; alguns algoritmos de ordenação podem ter uma complexidade próxima da quadrática (e alguns possuem uma complexidade próxima da logarítmica). Seguindo a mesma lógica, a leitura de um arquivo CSV



precisamos pensar na interação direta com o hardware (como gerenciar qual porta do hardware uma informação deve entrar ou sair, ou quais cálculos um processador precisa fazer para calcular novas colunas de um DataFrame).

No caso do `requests` e do `http.client`, pode ser que tenhamos uma aplicação que precisemos lidar com cookies e demais operações que geralmente fazemos pelo navegador: é muito mais fácil de fazê-lo via `requests` do que via `http.client`. Por outro lado, para algumas aplicações antigas, é provável que precisemos fazer ajustes finos que só seriam possíveis via `http.client`. Essa mesma tomada de decisão se aplica para outras bibliotecas e também para outras linguagens de programação.

Ao fazermos operações utilizando bibliotecas que já possuem uma lógica embutida para cálculos complexos, reduzimos tempo dos desenvolvedores. Uma ou duas linhas de código já podem gerar resultados nos quais teríamos que escrever dezenas ou até milhares de linhas de código em C para alcançar o mesmo resultado. Contudo, ao usarmos as bibliotecas, ficamos rendidos à lógica disponibilizada por elas: não há nada que comprove que uma determinada função de uma biblioteca em Python que encontramos na internet seja a mais rápida, a mais concisa ou a que contenha menos bugs – pelo contrário, é possível que certas implementações sejam desnecessariamente mais complexas do que outras ou, ainda, que contenham lógicas que raramente utilizaremos no nosso caso.

Saiba mais

Uma boa forma de entender a complexidade de algoritmos é ao visualizarmos algoritmos de ordenação, como a visualização do vídeo a seguir que compara 24 técnicas diferentes. Veja do 00:00 a 01:00. Existem algoritmos que precisam de muito mais tempo (e processador) para ordenar os mesmos dados.

Disponível em: <<https://www.youtube.com/watch?v=BeoCbJPuvSE>>. Acesso em: 29 nov. 2021.

TEMA 2 – CONCORRÊNCIA

Lembre-se de que, quando falamos de otimização de código, trouxemos à tona o aspecto do **tempo**. Gostaríamos que um algoritmo conduza o máximo de operações no menor tempo possível. Uma das formas de alcançarmos isso é



pela **paralelização** da operação: isto é, que não ocorra somente um cálculo ao mesmo tempo, mas vários em paralelo quando for possível.

Imaginemos um caso em que você se encontra sozinho preparando o seu jantar. Você está na cozinha e possui todos os utensílios à sua disposição. Uma das primeiras coisas que você deve supor (ou saber) é a de que você pode deixar mais de uma panela ligada ao mesmo tempo em um fogão caso esse possua mais de uma boca. Você poderia ter duas, três ou mais panelas ligadas ao mesmo tempo e todas elas cozinharão em **paralelo** os alimentos. Você também sabe que a quantidade de panelas que podem ser ligadas está diretamente ligada ao número de bocas disponíveis no fogão, e o tempo para cozinhar depende também do tipo de fogão e da panela: existem fogões industriais que podem preparar mais comida em menos tempo, por exemplo.

Além disso, você sabe que nem tudo deve ser paralelizado. É um pouco difícil mexer em mais de uma panela ao mesmo tempo, ou é impossível cortar alimentos e lavar os potes utilizados na preparação dos alimentos **ao mesmo tempo, sozinho**. Logo, você sabe que uma boa forma de preparar sozinho o seu jantar no menor tempo possível é planejando bem o passo a passo da preparação dos alimentos, a ordem das coisas que você precisa fazer e o que pode ser paralelizado. Como comentamos, é um trabalho essencialmente de **planejamento** e de **disponibilidade**. Também não adianta muita coisa você planejar que irá cozinhar três panelas ao mesmo tempo quando você só possui duas panelas disponíveis, não é?

A mesma lógica aqui se aplica para os algoritmos. Existem certas operações que podem ser executadas em paralelo, mas não todas. Para entender melhor isso, vamos primeiramente falar sobre **processos** e **threads**.

Um **processo** é um espaço no qual um determinado algoritmo é executado. Imagine um algoritmo que você construiu em Python para converter o seu nome para maiúsculas: se você executou o seu algoritmo hoje, esse criará um processo que executará em seu computador. Se executarmos o seu algoritmo ao mesmo tempo, mas em outro lugar, um outro processo será criado. Se você executar semana que vem, um terceiro processo será criado (e finalizado tão logo você complete a execução do algoritmo).

Logo, um processo é algo que pressupõe uma execução e que, por isso, possui três estados básicos: em execução; pronto para executar; e estado de espera. Existem processos que dependem mais de operações de processador



(como algoritmos que executam muitos cálculos e manipulações de variáveis) e processos que dependem da entrada e saída (*input* e *output*, ou I/O: aqui entram casos de leitura e escrita de arquivos, por exemplo).

É interessante também lembrar que é possível que um processo dê origem a **subprocessos**: por outro lado, cada processo possui um espaço reservado de consumo de memória; demora um pouco para ser criado e finalizado pelo sistema operacional; e não compartilha dados com outros processos. Para tentar endereçar esses problemas (principalmente o da lentidão para criar e finalizar processos, bem como a economia de recursos de um computador), criou-se o conceito de **thread**: um pequeno trecho de um algoritmo que pode ser executado junto com outros trechos de um mesmo processo – na analogia da cozinha que comentamos anteriormente, poderíamos entender que a thread é uma das panelas no fogo, e um processo assemelha-se a uma cozinha inteira (e, como toda analogia, ela não necessariamente é uma comparação perfeita).

Em Python, existem módulos em sua biblioteca padrão para o paralelismo baseado em threads, bem como para o paralelismo baseado em processos. Pensemos em um caso no qual precisemos fazer uma determinada operação que pode levar alguns segundos repetidas vezes:

```
import time
import random
import datetime
import threading
import concurrent.futures

def executar_funcionalidade(numero):
    now = datetime.datetime.now().isoformat()[-15:]
    print(f"Processando o número {numero} em {now}.")
    time.sleep(random.randint(1, 6))
    now = datetime.datetime.now().isoformat()[-15:]
    print(f"Finalizando o número {numero} em {now}.")
```

Esse código possui uma função chamada *executar_funcionalidade* a qual recebe um número qualquer. Após mostrar esse número na tela, geramos um número aleatório entre 1 e 6 (em amarelo). Se o número gerado foi 4, a thread ficará parada por 4 segundos (em verde). Se o número gerado foi 1, a thread aguardará por 1 segundo, por exemplo. Ou seja: independentemente do número



que foi apresentado a essa função, ela aguardará alguns segundos de forma aleatória.

Agora, vamos executar esta função usando threads: o código a seguir utiliza o `ThreadPoolExecutor` para executar repetidas vezes a função anterior. No caso, executaremos essa função 10 vezes (em amarelo) e disponibilizaremos 3 threads para executarem esta função em paralelo (em verde). Imagine como sendo dez clientes para serem atendidos em três caixas de supermercado: alguns clientes podem demorar mais do que os outros e, enquanto isso, os caixas trabalharão em paralelo. Assim que um cliente sair de um caixa, um próximo cliente será atendido até que os dez sejam atendidos.

```
# criando e inicializando as threads
with concurrent.futures.ThreadPoolExecutor(max_workers=3) as executor:
    executor.map(executar_funcionalidade, range(10))
```

```
Processando o número 0 em 17:02:38.293823.
Processando o número 1 em 17:02:38.296077.
Processando o número 2 em 17:02:38.297059.
Finalizando o número 0 em 17:02:44.302108.
Processando o número 3 em 17:02:44.302318.
Finalizando o número 1 em 17:02:44.302372.
Processando o número 4 em 17:02:44.302409.
Finalizando o número 2 em 17:02:44.302439.
Processando o número 5 em 17:02:44.302471.
Finalizando o número 5 em 17:02:45.303644.
Processando o número 6 em 17:02:45.303866.
Finalizando o número 3 em 17:02:46.304449.
Processando o número 7 em 17:02:46.304725.
Finalizando o número 4 em 17:02:48.306556.
Processando o número 8 em 17:02:48.306769.
Finalizando o número 6 em 17:02:50.309014.
Processando o número 9 em 17:02:50.309236.
Finalizando o número 7 em 17:02:51.309862.
Finalizando o número 8 em 17:02:53.311908.
Finalizando o número 9 em 17:02:55.314376.
```

Veja que os passos de finalização (em azul) não necessariamente seguem a mesma sequência da inicialização. O número 5, por exemplo, começou 17:02:55.302471 e terminou um segundo depois, em 17:02:45.303644. Se analisarmos bem os registros, veremos que nesse intervalo os números 3 e 4 ainda estavam sendo processados pelas outras threads. Assim que o número 5 foi finalizado, o número 6 tomou o seu lugar, e assim os números foram se alternando até a finalização do processamento.

Para usar múltiplos processos em Python, a sintaxe é parecida: ao invés de utilizarmos o `ThreadPoolExecutor`, utilizaremos o `ProcessPoolExecutor`. O



`max_workers` (em verde) agora refere-se ao número de processos, e não mais o de threads.

```
# criando e inicializando as threads
with concurrent.futures.ThreadPoolExecutor(max_workers=) as executor:
    executor.map(executar_funcionalidade, range(10))
```

```
Processando o número 0 em 17:09:22.578661.
Processando o número 1 em 17:09:22.580522.
Processando o número 2 em 17:09:22.584095.
Finalizando o número 2 em 17:09:23.594026.
Processando o número 3 em 17:09:23.606413.
Finalizando o número 1 em 17:09:24.591945.
Processando o número 4 em 17:09:24.594099.
Finalizando o número 0 em 17:09:25.589757.
Processando o número 5 em 17:09:25.592669.
Finalizando o número 4 em 17:09:25.598557.
Processando o número 6 em 17:09:25.601368.
Finalizando o número 5 em 17:09:27.597170.
Processando o número 7 em 17:09:27.599408.
Finalizando o número 3 em 17:09:27.612839.
Processando o número 8 em 17:09:27.615058.
Finalizando o número 6 em 17:09:28.606308.
Processando o número 9 em 17:09:28.610039.
Finalizando o número 8 em 17:09:28.618146.
Finalizando o número 7 em 17:09:32.606326.
Finalizando o número 9 em 17:09:33.619018.
```

Dica: uma sugestão é usar o `ProcessPoolExecutor` para casos nos quais podemos fazer uso de múltiplas cores no seu processador. O `ThreadPoolExecutor` pode ser uma boa sugestão para casos nos quais dependemos muito de entrada e saída de dados como leitura e escrita de arquivos ou requests para outros servidores.

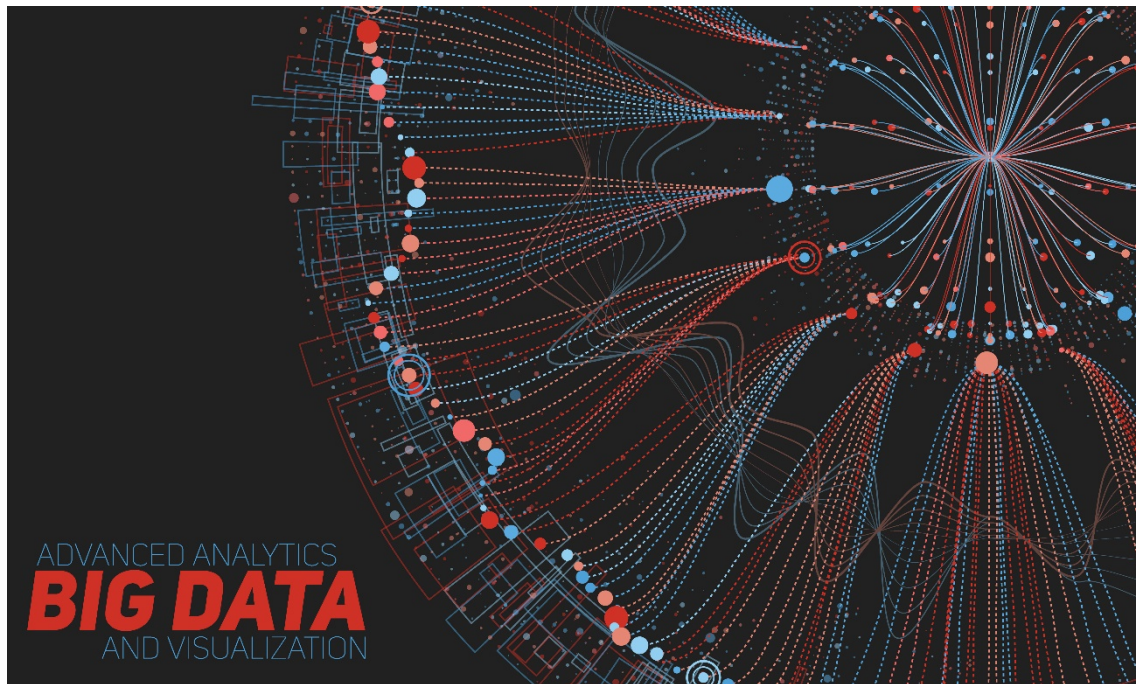
TEMA 3 – ESCALABILIDADE

A escalabilidade é um tema interessante no contexto de um aprendizado em uma Universidade. Pensemos juntos: se estamos na Universidade, buscamos aprender e dominar um conteúdo. Por outro lado, para aprender é importante colocar em prática o nosso conhecimento em um ambiente controlado e saber como atuar em cima de erros que possam aparecer. **Ambiente controlado** pode ser um tipo de situação-problema a qual sabemos a resposta e/ou que seja suficientemente simples para que consigamos responder em tempo hábil. É bem diferente de um problema real no qual há uma hierarquia administrativa (ou seja, um emprego com gestores e clientes) a qual lhe pressiona para entregar algo com prazos e sem espaço para erros: até porque errar em um emprego pode significar uma demissão ou um processo administrativo dependendo do tipo de trabalho desenvolvido.



Já na Universidade, podemos saber em que erramos e em que poderíamos melhorar. No pior dos casos, poderíamos perder um pouco de nota (mas ainda sim teríamos um feedback para saber se estamos indo bem ou não: a nota é um indicativo disso). É nesse sentido que muitos dizem que a Universidade é o local ideal para errar, já que fora dela o risco é bem maior.

Por outro lado, pela própria natureza dos problemas sendo estudados, é um potencial risco acabarmos sendo mal-acostumados às situações-problema da Universidade: algoritmos relativamente simples; trabalhos individuais ou em grupos pequenos; dados controlados e, no máximo, o nosso código precisa funcionar no nosso computador e talvez em mais um ou outro. O código será produzido por alguns dias ou semanas, tempo suficiente para que entreguemos a versão do código e que, no final das contas, ele seja avaliado e que recebamos uma nota como resultado.



Crédito: GarryKillian/shutterstock.

Isso, naturalmente, pode levar ao problema de não nos depararmos com uma situação bem menos elegante no mundo real: algoritmos com dezenas ou centenas de milhares de linhas e escritos por várias pessoas com padrões diferentes; códigos utilizando padrões antiquados ou com redundâncias desnecessárias; códigos sem documentação ou com a documentação incorreta; entre outros. Esses códigos possuem a tendência de serem muito mais complexos do que aquilo que estávamos acostumados a ver na Universidade e



muito mais críticos em sua aplicação. Logo, qualquer mudança pode ser muito mais arriscada.

TEMA 4 – CPYTHON

Primeiramente: existe o **CPython** e o **Cython** (além do Python, claro). E ambos se referem a coisas completamente diferentes. O **Cython** é uma linguagem de programação que *inclui* o que geralmente desenvolvemos em Python, mas *adiciona* algumas características do C. Os criadores do Cython comentam que ele não é um “tipo” de Python (ou seja, uma implementação dele), mas que ele deve ser capaz de rodar códigos em Python.

Dito isso, o foco aqui é no **CPython**. É extremamente provável que todo o código em Python que rodamos até agora utilizou o CPython por trás dos panos. Na sua formação, é importante que saiba a diferença entre linguagem e implementação. Vamos lá.

1. Uma **linguagem** (e isso inclui o Python, Java, C#, C, C++, Swift e várias outras) é, de forma sucinta, um conjunto de sintaxe e regras para escrevermos códigos que serão um software. Se você escrever à mão com uma caneta em um guardanapo um código que utiliza a sintaxe em Python, você terá utilizado com sucesso o seu conhecimento para escrever um código válido em Python. Por outro lado, por ter feito em um guardanapo à caneta não significa necessariamente que esse código irá *funcionar*, não é mesmo?
2. Já a **implementação** é aquilo que fará com que o seu código funcione. É ela quem receberá o código que você ou outras pessoas escreveram e, se estiver tudo certo com o código, executará com sucesso as instruções. Geralmente as implementações precisam ler, interpretar (ou compilar) o código e fazer com que ele funcione corretamente. Uma determinada linguagem pode ter várias implementações, mas entre elas haverá uma que é utilizada como referência por todas as demais; uma em que a maioria das pessoas utilize (até mesmo sem saber que a estão utilizando).

Quando vamos no site oficial do Python para baixá-lo, nós veremos que ele, no final das contas, utiliza o **CPython**. Logo, ele é a implementação de referência do Python. Como mencionamos anteriormente, até agora é provável que todos (ou quase todos) os seus códigos utilizaram o CPython sem saber: ele



é a implementação tradicional do Python e foi escrito com as linguagens de programação Python e C.

Existem outras implementações em Python como:

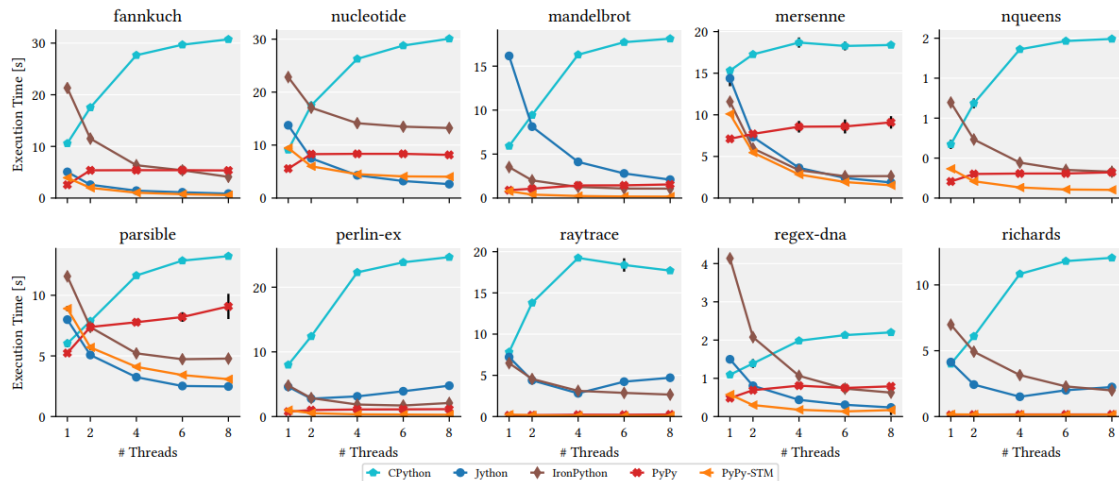
- PyPy, o qual utiliza um compilador em tempo de execução (ou seja: compila o código enquanto o algoritmo está sendo executado ao invés de fazê-lo *antes* da sua execução);
- IronPython, o qual utiliza o .NET, um framework da Microsoft; e
- Jython, o qual utiliza Java.

Cada uma das implementações pode converter o seu código escrito em Python em um resultado. Você poderá escrever um código que executa uma multiplicação entre dois números e as implementações irão dar conta do seu código e converter isso em um resultado. A diferença está na **forma** na qual elas fazem isso: às vezes, um determinado código pode ser mais rápido em CPython do que em PyPy, e vice-versa – ainda que ambos produzam exatamente o mesmo resultado. O segredo está na forma na qual esses códigos otimizam a sua lógica e utilizam os recursos do seu computador. Pensemos no exemplo de dois carros que possuam o mesmo preço, dimensões e tipo de motor, mas produzidos por fabricantes diferentes: um deles pode consumir mais combustível do que o outro, mas pode se sair melhor no conforto ou nas retomadas – o segredo dessas diferenças pode estar na mecânica ou nos materiais empregados por ambos os carros. Ou seja, não está exatamente necessariamente no jeito que dirigimos, mas sim no que está “por trás dos panos” em ambos os carros. A mesma lógica se aplica para essas diferentes implementações.

Além disso, dependendo da sua pesquisa e do seu interesse, você pode ter se deparado com o argumento de que “Python é lento”. Alguns estudos (disponíveis nas referências bibliográficas desta aula) mostram que o Jython e IronPython possuem uma performance para operações sequenciais (isto é, quando **não usamos** todos os threads disponíveis em paralelo de um processador) parecida com o CPython, mas são até 3 vezes mais rápidos para operações em paralelo. Ou seja: a implementação de referência (que é o CPython) pode ser mais **lenta** do que as demais, o que poderia justificar a afirmação anterior. Além disso, alguns estudos experimentais considerando sistemas críticos (isto é, sistemas que precisam de uma taxa muito baixa de erros



como sistemas de segurança ou que sustentam as operações de grandes indústrias) mostraram, por exemplo, que o PyPy pode ser mais **confiável** que o CPython, o qual, por sua vez, poderia ser mais confiável do que o Jython sob o contexto deste tipo de sistema. Então, por que mesmo assim muitos utilizam o CPython?



Fonte: Meier; Gross. Reflections on the Compatibility, Performance, and Scalability of Parallel Python (2019. p. 91-103).

A resposta mais simples para isso fica no próprio fato do CPython ser a implementação de referência do Python: novas funcionalidades do Python serão disponibilizadas primeiramente no CPython do que em outras implementações e, ainda, é mais provável que os módulos funcionarão sem problemas no CPython do que em outras implementações – logo, há o risco da (in)compatibilidade do seu código dependendo dos módulos em uso.

TEMA 5 – PROTOCOLOS E INTERFACES

Um dos aspectos que tornam este estudo especial é a junção e a aplicação de diferentes temas. Isto é, ainda que possamos dividir o conteúdo de nossos estudos em diferentes aulas e temas, na prática poderíamos reordená-la sem grandes prejuízos à nossa aprendizagem. Tudo o que aprendemos sobre bibliotecas, programação orientada a objetos e arquivos fazem parte de uma grande temática a qual nos capacita para o desenvolvimento de algoritmos complexos que resolvem problemas de negócio.

Um dos temas que também fazem parte da programação orientada a objetos possui o nome de **interfaces**, ou **protocolos** (dependendo da linguagem de programação). Em Python, chamamos de *protocolo*. Imagine que os



protocolos servem para garantir e reforçar algumas propriedades que uma classe que a utiliza deva seguir. Caso essa frase tenha ficado confusa, pensemos em um exemplo não muito legal. “Fomos fritar um ovo e o resultado não foi o esperado”, como você pode ver na imagem a seguir:



Crédito: mon_ter/shutterstock.

Para nossa sorte, você estava do nosso lado para nos ajudar. Pedimos a sua ajuda, e você me pergunta como poderia me ajudar. Desesperado, respondemos a você: “qualquer coisa para jogar aqui no fogo!”. Agora, o “qualquer coisa” pode incluir os seguintes itens:

- um extintor;
- outra panela;
- um travesseiro;
- um cobertor;
- uma garrafa de álcool;
- uma boa quantidade de areia;
- uma bexiga de aniversário; e
- um óculos.

Naturalmente, alguns desses itens não ajudariam muito. Logo, não é exatamente “qualquer coisa”, e sim “algo específico que apague o fogo”. Até



poderíamos ser mais específicos e falar “algo líquido que apague o fogo” ou “algo sólido que apague o fogo”. O ponto aqui é o seguinte: às vezes, o que temos na nossa cabeça e entendemos como óbvio pode não ser tão óbvio para você, e vice-versa. Sabemos que esse exemplo beira o absurdo, mas em TI podemos nos deparar em alguns casos tão extremos quanto. Um outro desenvolvedor pode acabar sendo “criativo” demais e reinventando coisas que já estavam implementadas; podem acabar saindo do padrão pré-determinado pela empresa sobre a nomenclatura ou o tipo de dados que poderia (ou deveria) ser manipulado, e assim por diante. Logo, as interfaces ajudam nesse trabalho de **padronização** e **garantia** de que tudo o que deva ser implementado pelas classes será, de fato, implementado.

Agora, de uma forma um pouco mais técnica: o protocolo (ou interface: veja que estamos de propósito utilizando ambos os termos para que fique claro para você que, dependendo da linguagem de programação, um ou outro são usados) oferece uma forma de prover uma **abstração** ao seu código: você define **quais são** os métodos que uma classe deve ter, mas **não determina o código** que esses métodos devem ter.

Os protocolos estão disponíveis a partir do Python 3.8 e os definimos como uma classe que herda o Protocol do módulo typing. Dentro dessa classe (que na verdade é o nosso protocolo/interface), estabelecemos quais são os métodos que uma classe que a utilize deva ter por padrão. A linha realçada significa que, por padrão, ela gerará um erro toda vez que for utilizada, a não ser que implementemos posteriormente alguma lógica para substituí-la. Além disso, perceba o “-> str”. Isso indica que é esperado que este método retorne uma string.

```
from typing import Protocol

class Animal(Protocol):
    def obter_nome(self) -> str:
        raise NotImplementedError
```

E como uma classe utilizaria isso? Pensemos na classe *Cachorro* que vimos anteriormente. Observe que ela não está herdando o protocolo *Animal*. Ao tentarmos chamar o método *obter_nome*, teremos um erro:



```
class Cachorro:
    def __init__(self, nome: str, idade: int):
        self.__nome = nome
        self.__idade = idade
```

```
pippoca = Cachorro('Pippoca', 9)
pippoca.obter_nome()
```

```
AttributeError                                Traceback (most recent call last) in
<module>
      5
      6 pippoca = Cachorro('Pippoca', 9)
----> 7 pippoca.obter_nome()
```

```
AttributeError: 'Cachorro' object has no attribute 'obter_nome'
```

Esse erro significa que ele não encontrou nada chamado “obter_nome” na classe *Cachorro* e, por isso, não soube o que fazer. Vamos modificar a classe cachorro para que ela herde o protocolo *Animal*:

```
class Cachorro(Animal):
    def __init__(self, nome: str, idade: int):
        self.__nome = nome
        self.__idade = idade
```

```
pippoca = Cachorro('Pippoca', 9)
pippoca.obter_nome()
```

```
AttributeError                                Traceback (most recent call last) in
<module>
      5
      6 pippoca = Cachorro('Pippoca', 9)
----> 7 pippoca.obter_nome()
```

```
NotImplementedError:
```

O erro agora é diferente. Na realidade, ele está retornando o comportamento padrão do protocolo que é aquele “*raise NotImplementedError*” que vimos anteriormente. Como desenvolvedores, isso funciona um alerta para nós: *Cachorro* é um tipo de *Animal* e obrigatoriamente devemos colocar alguma lógica para um método chamado *obter_nome*. Se não fizermos isso, poderemos ter problemas em outras partes do código em que é esperado que utilizemos



esse método. Logo, basta implementarmos esse método que resolveremos essa situação:

```
class Cachorro(Animal):  
    def __init__(self, nome: str, idade: int):  
        self.__nome = nome  
        self.__idade = idade  
  
    def obter_nome(self):  
        return self.__nome
```

```
pippoca = Cachorro('Pippoca', 9)  
pippoca.obter_nome()
```

```
'Pippoca'
```

E é isso! Implementamos com sucesso um caso de uso com protocolos em Python.

FINALIZANDO

Nesta aula, tivemos a oportunidade de observar alguns tópicos mais próximos à arquitetura e à melhoria de algoritmos: no processo de desenvolvimento de software, podemos nos deparar com casos em que um determinado algoritmo que funciona muito bem em nosso computador pode ter uma performance abaixo do esperado em outros casos de uso – ou, ainda, pode ser que em determinados casos alguém escolha uma determinada biblioteca ou linguagem de programação sem uma justificativa técnica em si. Esse tipo de entendimento possui alguns riscos associados, como termos pré-conceitos de que uma linguagem é melhor do que a outra em todos os casos possíveis ou que uma biblioteca é melhor do que a outra pelo mesmo motivo. Entender os conceitos de escalabilidade, otimização e afins ajuda nesse sentido.

Ainda, vimos sobre o uso das interfaces (ou protocolos). Elas são úteis no paradigma de programação orientada a objetos por especificarem os métodos que devem ser implementados sem que necessariamente os implementemos: logo, ajudam muito na padronização do código e no projeto de uma boa solução de software.

Esses conteúdos, em conjunto com o que abordamos nas demais aulas, formam um material robusto sobre linguagem de programação que é essencial



para o desenvolvimento de algoritmos de elevada qualidade. Observe que em nenhum momento realizamos um guia em formato de passo a passo, mas sim fomos analisando diferentes comportamentos e implementações de código: a intenção aqui não é a de que você somente execute códigos, mas, sim, que compreenda as diferenças e as consequências do uso (ou não uso) de determinadas escolhas.



REFERÊNCIAS

- AGUILAR, L. J. **Fundamentos de programação**: algoritmos, estruturas de dados e objetos. 3. ed. Porto Alegre: AMGH, 2011.
- CERVEIRA, F. et al. Soft error sensitivity and vulnerability of languages and their implementations. **International Journal of Critical Computer-Based Systems**, v. 9, n. 4, p. 318-347, 2019.
- HIRAMA, K. **Engenharia de Software**: qualidade e produtividade com tecnologia. Rio de Janeiro: Elsevier, 2011.
- MEIER, R.; GROSS, T. R. Reflections on the compatibility, performance, and scalability of parallel Python. In: **Proceedings of the 15th ACM SIGPLAN International Symposium on Dynamic Languages**. 2019. p. 91-103.
- MACHADO, F. B.; MAIA, L. P. **Arquitetura de Sistemas Operacionais**. Rio de Janeiro: LTC, 2017.
- PERKOVIC, L. **Introdução à Computação Usando Python**: um Foco no Desenvolvimento de Aplicações. Rio de Janeiro: LTC, 2016.
- SCHACH, S. R. **Engenharia de Software**: os paradigmas clássico e orientado a objetos. Porto Alegre: AMGH, 2010.
- TOSCANI, L. V.; VELOSO, P. A. S. **Complexidade de Algoritmos**: análise, projeto e métodos. Porto Alegre: Bookman, 2012.